제목: 비전공자도 바로 쓰는 Git & GitHub 첫걸음

부제: 버전 관리의 개념부터 최신 필수 명령어까지

임성국

화면캡처

WIN + Shift + S

Git vs GitHub

[파일 목록]

보고서_최종.docx,

보고서_진짜최종.docx,

보고서_진짜진짜_최종_수정.docx

- ❑ 핵심 키워드: 세이브 포인트(Check Point), 타임머신, 버전 관리 시스템(VCS)

Git
➔ "내 컴퓨터 안의 작업 기록 엔진 (오프라인 가능)"

GitHub
➔ "클라우드 저장소 & 협업 플랫폼 (온라인 웹 서비스)"

깃의 3단계 흐름 (작업실 ➔ 바구니 ➔ 금고)

다이어그램:

- Working Directory (작업 공간) ➔
- git add ➔
- Staging Area (준비 영역) ➔
- git commit ➔
- Repository (저장소/금고)

작업 시작과 저장

- 작업 시작하기 (Init & Status)
  - git init : 현재 폴더를 Git 저장소로 등록
  - git status : 변경된 파일 상태 점검 (가장 중요한 나침반)
- 담기와 저장하기 (Add & Commit)
  - git add <파일명> / **git add .** : 특정 파일 또는 현재 폴더 전체 담기
  - **git add --all** (또는 -A) : 전체 프로젝트 변경사항 담기
  - **git commit -m "작업 내용 요약"** : 세이브 포인트 확정

기록 확인하기 (Log)

- git log : 전체 커밋 상세 이력 확인
- git log --oneline : 한 줄씩 깔끔하게 요약해서 보기
  - 화면 예시: 커밋 해시(고유 ID)와 커밋 메시지 목록

GitHub와 연결하고 첫 업로드하기 (Remote & Push)

- 1단계: git branch -M main (기본 브랜치 이름 통일)
- 2단계: git remote add origin <GitHub_저장소_URL> (원격 주소 등록)
- 3단계: git push -u origin main (첫 업로드 및 연결 설정)
  - 이후부터는 단순히 git push만 입력하면 끝!

협업과 내려받기 (Clone & Pull)

- git clone <URL> : 원격 저장소 통째로 복제해오기 (최초 1회)
- git pull : 원격 저장소의 최신 변경사항만 쏙 내려받기 (수시로)

최신 브랜치(Branch) 활용법

- 브랜치 개념도: 원본 줄기(main)에서 뻗어 나오는 나뭇가지(feature/login)
- git switch -c <새브랜치> : 브랜치 생성과 동시에 이동 (최신 표준)
- git switch <브랜치> : 기존 브랜치로 이동

5대 핵심 루틴

- 상태 확인: `git status`
- 바구니 담기: `git add .`
- 세이브 찍기: `git commit -m "작업 내용"`
- 최신화 당기기: `git pull`
- 클라우드 전송: `git push`

좋은 커밋 습관

- 좋은 습관 1: 커밋 메시지는 '무엇을, 왜' 했는지 구체적으로 적기
  (❌ "수정", ⭕ "로그인 버튼 클릭 오류 수정")
- 좋은 습관 2: 비밀번호나 API 키는 절대 깃에 올리지 않기
  (.gitignore 파일 활용)

깃 & 깃허브 명령셋 샘플

내 PC 의 c:\dev\project-ds 를 깃으로 저장하고,

최종 완성된 것을 github 의 github.com/jacky 의 prj-ds 리포지토리에 연결한다.

1단계: 사용자 기본 정보 설정(1회만 설정)

```
git config --global user.name "jacky"
git config --global user.email "jacky@example.com"
```

2단계: 프로젝트 폴더 이동(파일탐색기 - cmd)

```
cd c:\dev\project-ds
```

3단계: 로컬 깃 초기화 및 첫 커밋 (Save Point 찍기)

```
git init    # 1. 현재 폴더를 Git 저장소로 초기화
git status      # 2. 현재 상태 점검 (README.md, main.py가 빨간색으로 표시됨)
git add .       # 3. 모든 파일을 장바구니(Staging Area)에 담기  (또는 git add --all)
git commit -m "feat: 첫 번째 커밋 (프로젝트 초기 세팅)"         # 4. 첫 번째 커밋 확정
git log --oneline       # 5. 커밋이 잘 되었는지 한 줄 로그로 확인
```

4단계: 브랜치 이름 통일 (main 표준화)

```
git branch -M main
```

5단계: 원격 저장소(GitHub) 연결

```
git remote add origin https://github.com/jacky/prj-ds.git   # 연결이 잘 되었는지 주소 확인
git remote -v
```

6단계: GitHub로 첫 업로드 (Push)

```
git push -u origin main
```

7단계: 최종 확인

```
git status
```

[2번째 접속 이후]

```
> git  add  .
> git  commit  -m  " My Comment "
> git  push
```

[수정 및 보완 팁] 데이터 사이언스(project-ds) 프로젝트 필수 권장사항

```
# .gitignore 파일 (가상환경, 주피터 체크포인트, 이미지나 동영상 파일, 대용량 csv파일 제외 설정)
    echo "venv/" >> .gitignore
    echo ".env" >> .gitignore
    echo "*.jpg" >> .gitignore
    echo ".ipynb_checkpoints/" >> .gitignore
    echo "*.csv" >> .gitignore

# 새로 추가된 .gitignore 파일 저장 및 업로드
    git add .gitignore
    git commit -m "chore: gitignore 파일 추가"

# 이후부터는 단순하게 git push만 입력하면 자동 업로드됩니다.
    git push
```

```
venv/
.env
*.jpg
.ipynb_checkpoints/
*.csv
```

END

필수 설치와 설정

1. git 다운로드 & 설치 (option 변경없이 그대로)
2. github 계정 만들기
3. 내 PC 에서 프로젝트 만들고 저장
   a. git config --global user.name "jacky"
   b. git config --global user.email "jacky@example.com"

   c. cd c:\dev\project-ds # 이동
   d. git init   # 1. 현재 폴더를 Git 저장소로 초기화
   e. git add .   # 3. 모든 파일을 장바구니(Staging Area)에 담기 (또는 git add --all)
   f. git commit -m "feat: 첫 번째 커밋 (프로젝트 초기 세팅)"   # 4. 첫 번째 커밋 확정
   g. git branch -M main
   h. git remote add origin https://github.com/jacky/prj-ds.git   # 연결이 잘 되었는지 확인
   i. git push -u origin main
4. **cmd**

[2번째 접속 이후]

```
> git  add  .
> git  commit  -m  " My Comment "
> git  push
```